Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

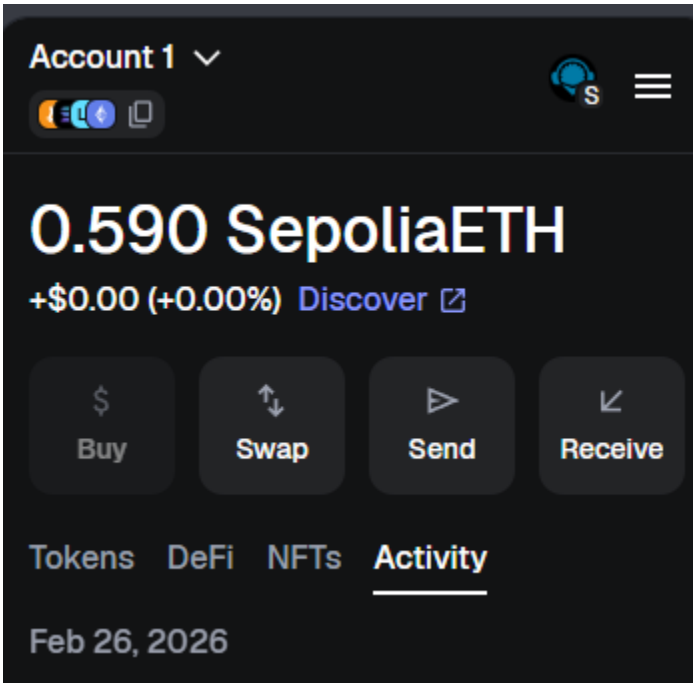
Requirements

- MetaMask installed
- Sepolia network enabled
- Sepolia ETH in **Account 1** (deployer) and ideally **Account 2** (receiver)
- Remix IDE: remix.ethereum.org
- Your ERC1155 contract code

Part A — MetaMask Sepolia Setup

1. MetaMask → enable **Test Networks**
2. Select **Sepolia**
3. Get Sepolia ETH from a faucet
4. Make sure you have enough for deployment + transactions

Checkpoint: You have Sepolia ETH in Account 1

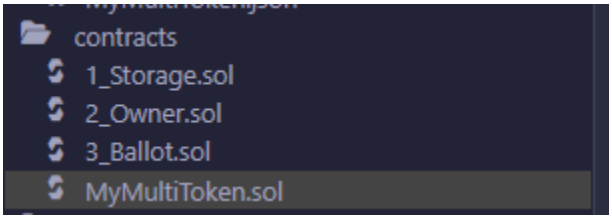


Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

Part B — Remix Setup

1. Open Remix
2. Create file: MyMultiToken.sol
3. Paste your ERC-1155 contract code

Checkpoint: file saved in Remix



Part C — Fix OpenZeppelin Imports (if needed)

1. Go to **Solidity Compiler** → Compile once
2. If imports fail:
    - Add OpenZeppelin contracts using Remix Dependencies / Packages

Checkpoint: *no import errors*

Part D — Compile

1. Compiler version: **0.8.x** compatible with ^0.8.0
2. Click **Compile**

Checkpoint: *compilation success*

Part E — Connect Remix to MetaMask (Sepolia)

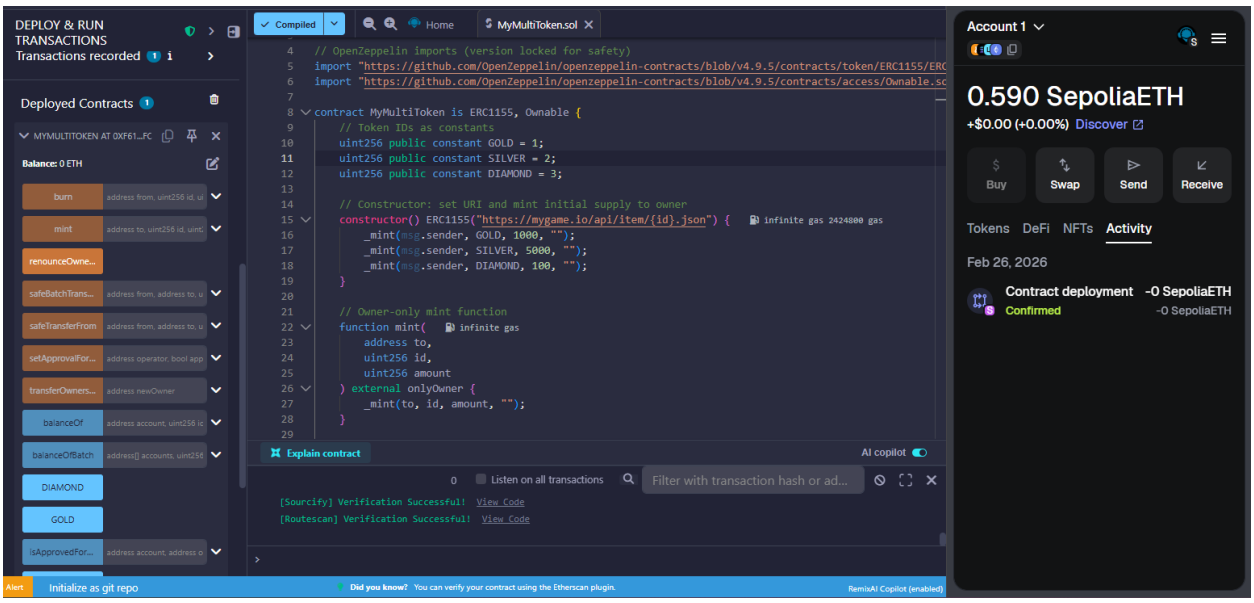
1. Deploy & Run Transactions tab
2. Environment: **Injected Provider (MetaMask)**
3. Approve MetaMask connection
4. Confirm network is **Sepolia**

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301
Checkpoint: *Remix shows your MetaMask account*

Part F — Deploy Contract

1. Select contract: MyMultiToken
2. Click **Deploy**
3. Confirm in MetaMask

Checkpoint: *contract appears under Deployed Contracts*



Part G — Read & Verify Initial Supplies

Test G1: Check initial balances (Account 1)

In the deployed contract UI, call:

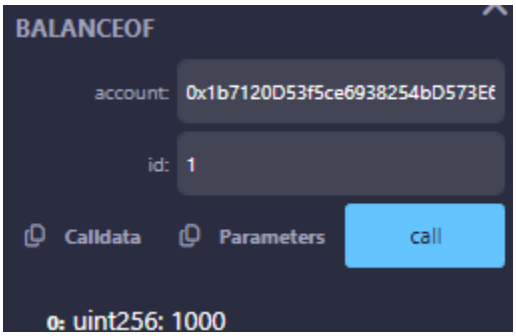
- balanceOf(account1, 1) → should be **1000**
- balanceOf(account1, 2) → should be **5000**
- balanceOf(account1, 3) → should be **100**

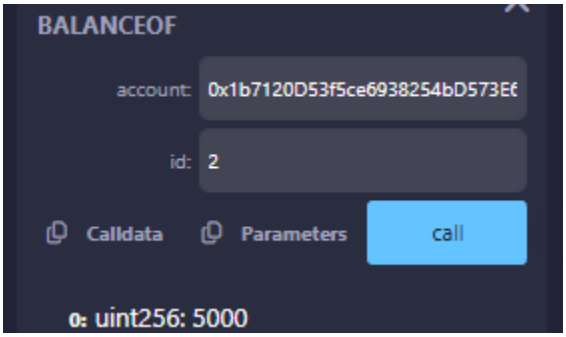
Checkpoint: balances match the constructor mints

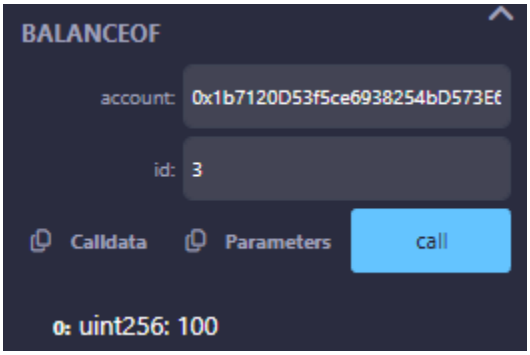
If your UI shows a balanceOf function:

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301
Enter:

- account: your address
- id: 1, 2, or 3







Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

Part H — Test Owner-only Mint

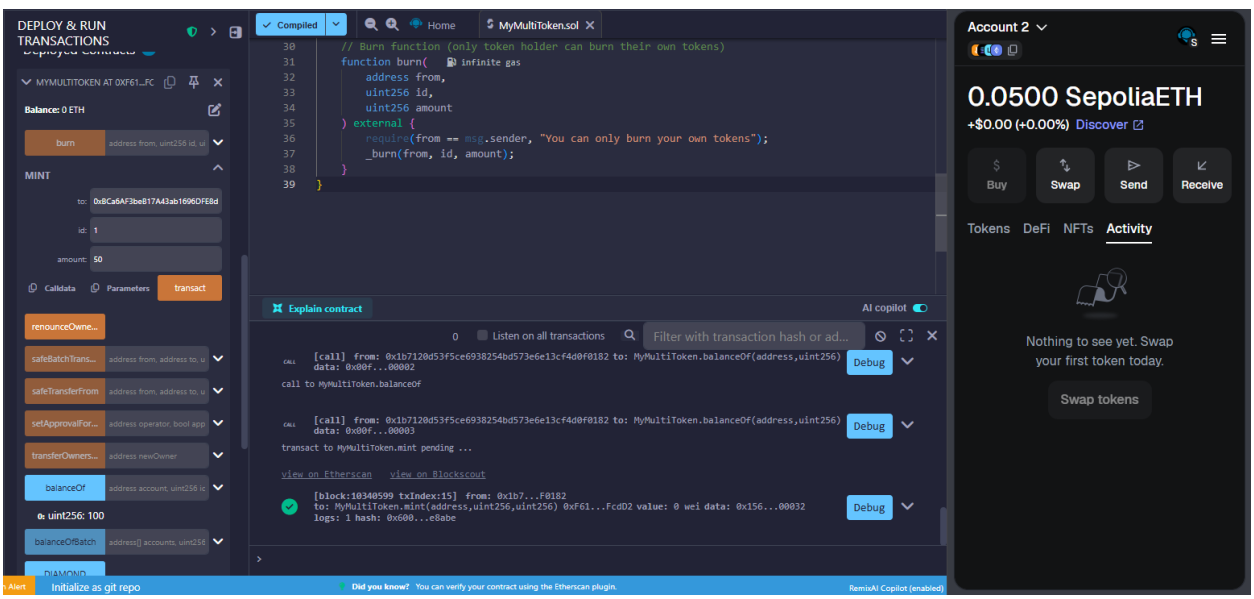
Test H1: Mint GOLD to Account 2 (owner action)

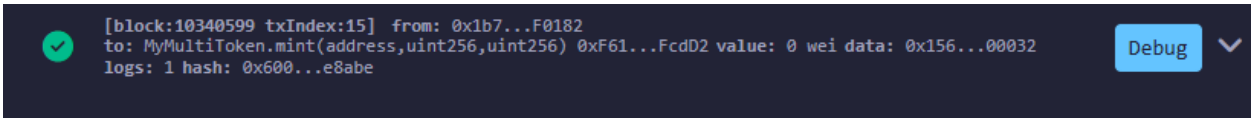
1. Switch nothing (stay in Account 1 owner)
2. Call: mint(to=Account2, id=1, amount=50)
3. Confirm transaction

Then verify:

- balanceOf(Account2, 1) → **50**
- balanceOf(Account1, 1) → still **1000** (because mint creates new tokens)

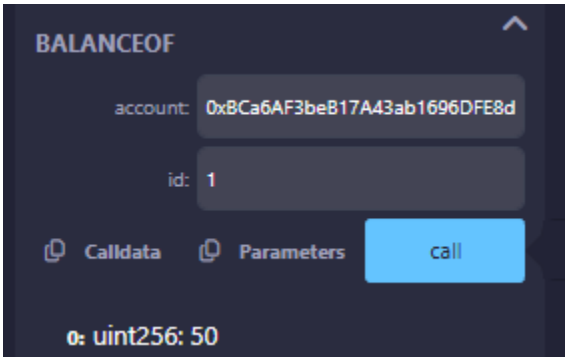
Checkpoint: *Account 2 received minted GOLD*



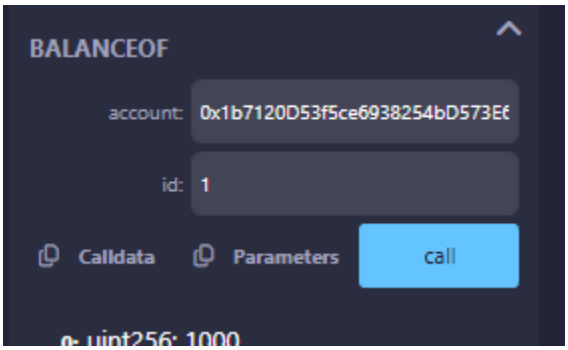


Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

BALANCE OF ACCOUNT 2:



BALANCE OF ACCOUNT 1:



Test H2: Non-owner tries to mint (should FAIL)

1. Switch MetaMask to **Account 2**
2. In Remix, still Injected Provider (it follows MetaMask active account)
3. Call: mint(to=Account2, id=2, amount=10)

Expected: *Transaction **reverts** (fails) due to onlyOwner*

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

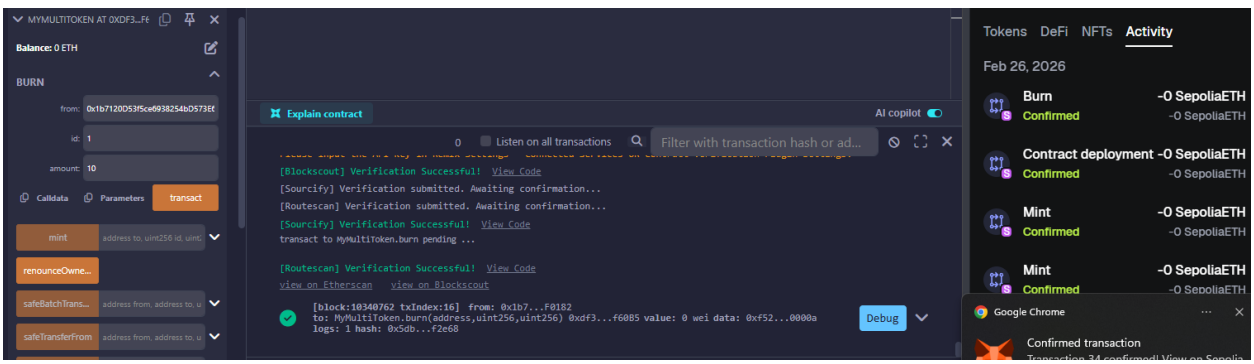
Part I — Burn Tests (Self-only)

Test I1: Burn your own tokens (Account 2)

1. Keep MetaMask on **Account 2**
2. Call: burn(from=Account2, id=1, amount=10)
3. Confirm transaction

Verify: balanceOf(Account2, 1) decreased from 50 → 40

Checkpoint: burn works for your own balance



Test I2: Attempt to burn someone else's tokens (should FAIL)

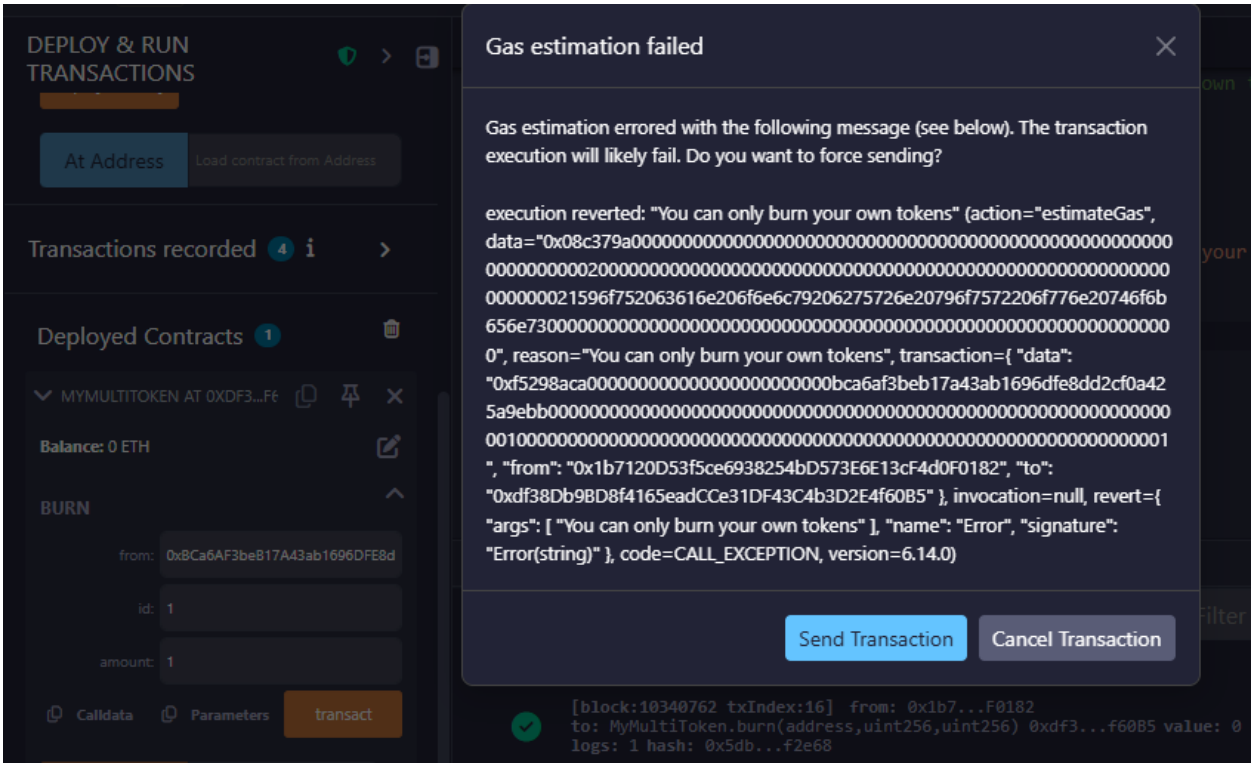
Still in Account 2:

- call burn(from=Account1, id=1, amount=1)

Expected: Revert with:

- **"You can only burn your own tokens"**

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301



Part J — Transfer Tests (ERC-1155 Safe Transfer)

Test J1: Transfer GOLD from Account 1 to Account 2

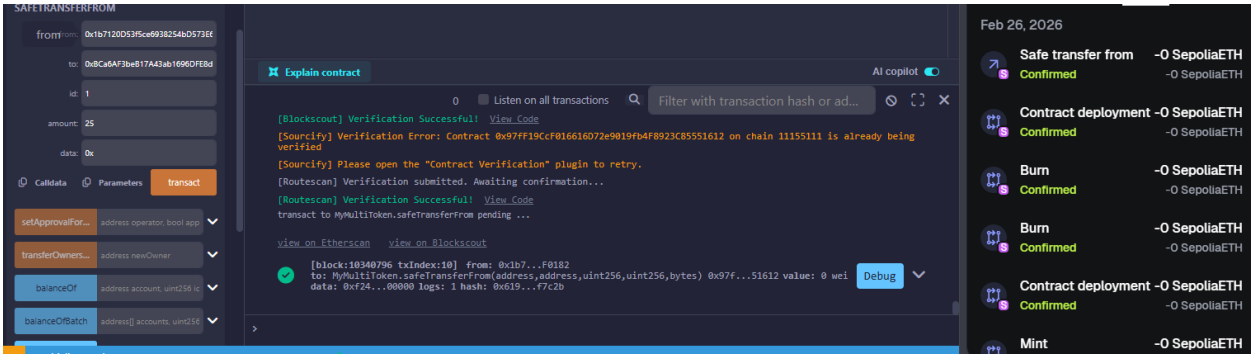
1. Switch MetaMask back to **Account 1**
2. Use: safeTransferFrom(from=Account1, to=Account2, id=1, amount=25, data=0x)
3. Confirm transaction

Verify:

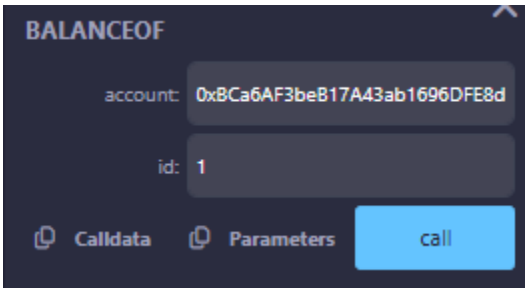
-
  -
    - balanceOf(Account2, 1) increases by 25
    - balanceOf(Account1, 1) decreases by 25

Checkpoint: *ERC1155 transfer works*

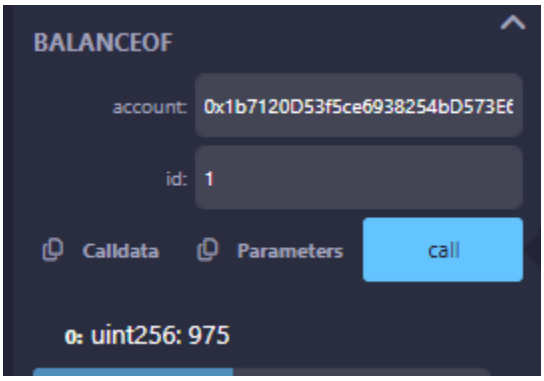
Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301



ACCOUNT 2:



ACCOUNT 1



Part K — Approval Tests (Operator Approval)

ERC-1155 supports "operators" who can transfer on your behalf:

- setApprovalForAll(operator, approved)
- isApprovedForAll(owner, operator)

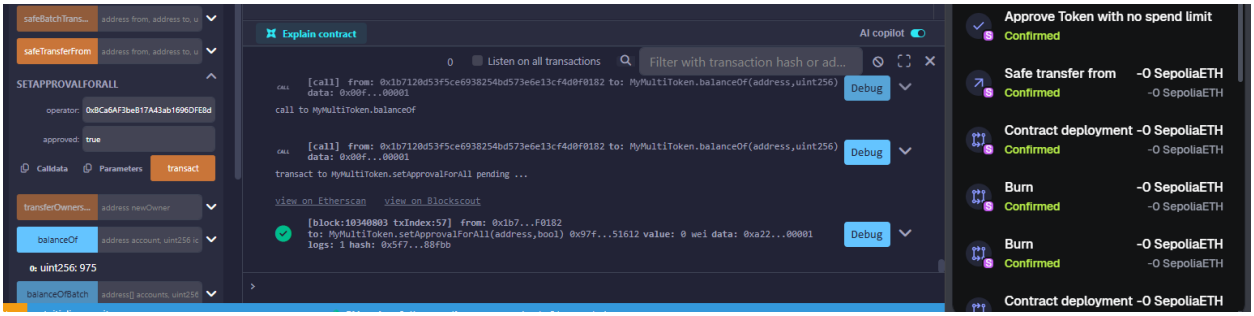
Test K1: Approve Account 2 as operator for Account 1

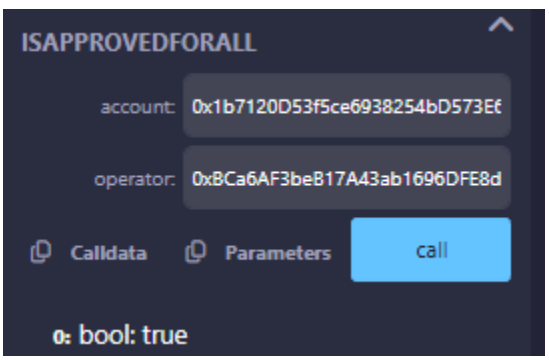
1. MetaMask: Account 1

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

2. Call:
    - setApprovalForAll(Account2, true)
3. Verify:
    - isApprovedForAll(Account1, Account2) → **true**

Checkpoint: operator approval enabled



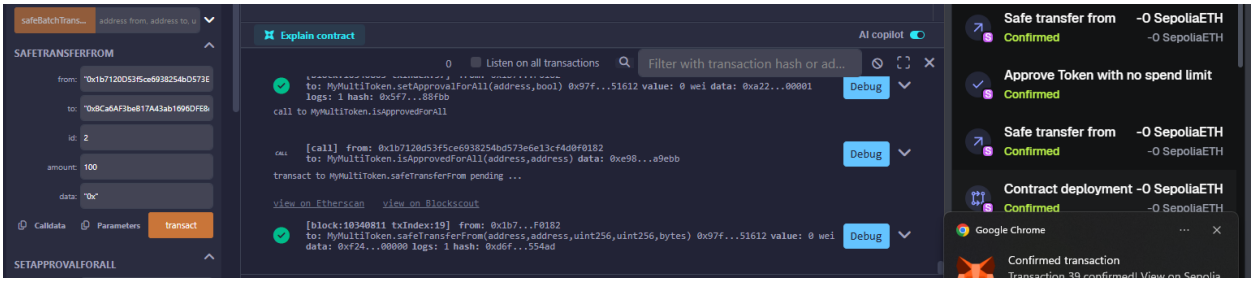


Test K2: Transfer from Account 1 using Account 2 as operator

1. Switch MetaMask to Account 2
2. Call:
    - safeTransferFrom(from=Account1, to=Account2, id=2, amount=100, data=0x)

Expected: should succeed **only if approval is true**

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301



---

4) Common gotchas (ERC-1155 specific)

- **URI doesn't open**: your URI is a sample domain; it won't resolve unless you host metadata.
- **{id} formatting**: ERC-1155 expects {id} as 64-hex padded by many implementations; OpenZeppelin follows the standard substitution behavior.
- **Burn restriction**: your burn requires from == msg.sender — operators cannot burn on behalf (unless you change logic).
- **Transfers require data bytes**: in Remix, use 0x or 0x00... as needed.

---

5) Post-lab questions (ERC-1155 version)

1. What are the token IDs for GOLD, SILVER, DIAMOND?

   uint256 public constant GOLD = 1;

   uint256 public constant SILVER = 2;

   uint256 public constant DIAMOND = 3;

2. Why can Account 2 not call mint()?

   Account 2 cannot call mint() because the function is restricted by the onlyOwner modifier. Only the contract owner (Account 1, the deployer) is allowed to mint new tokens, so any attempt by Account 2 will revert.

3. What function confirms how many tokens of an ID a wallet owns?

Cortez, Lawrence Neil M.
Macaraeg, Alvin
NW-301

The function that confirms how many tokens of a specific ID a wallet owns is balanceOf(address account, uint256 id). It requires both the wallet address and the token ID to return the balance.

4. Why does burn(Account1, id, amount) fail when called by Account 2?

   The call burn(Account1, id, amount) fails when executed by Account 2 because the burn function requires that from must be equal to msg.sender. This means users can only burn their own tokens, not someone else's.

5. What is the difference between _mint() and safeTransferFrom()?

   The difference between _mint() and safeTransferFrom() is that _mint() creates new tokens and increases supply, while safeTransferFrom() transfers existing tokens from one address to another without creating new tokens.